

1 Introduction

This report presents an analysis of different methods for parallelising code that computes probabilistic statistics related to the connectivity of large random graphs. The program calculates the average number of connected components and the size of the largest connected component, averaging these values across a fixed number of graphs. Each graph is composed of `Nvert` vertices, with a specified probability determining the likelihood of an edge existing between any two vertices.

We begin by running the original code on `vonneumann` without any parallelism and using the parameters `Np = Ngraphs = 1`. This produces the following output:

Output
Pcon = 0.0004 Av. Num. Clusters. = 805.0000 Av. Largest Cluster = 3943.0000 Time elapsed : 0.0843 seconds

Table 1: Summary of original code results with no parallelism

Next, we enable parallelism in the original code by adding the `-fopenmp` flag to the Makefile. We then compile and run the code with OpenMP, which includes the existing data race issues, using the default setting (`export OMP_NUM_THREADS=32`) on `vonneumann`. After 50 iterations, we observe two distinct outputs, as shown below:

Threads	Output
32 (default)	Pcon = 0.0004 Av. Num. Clusters. = 840.0000 Av. Largest Cluster = 3949.0000 Time elapsed : 0.0339 seconds
32 (default)	Pcon = 0.0004 Av. Num. Clusters. = 841.0000 Av. Largest Cluster = 3948.0000 Time elapsed : 0.0354 second

Table 2: Two distinct results due to a present data race in parallel region

One observes that after a few runs of the original code, we have two distinct values for the number of clusters. This is caused by the data race present.

1.1 Parallelism over vertices

Firstly, we note that the variables `i`, `j`, `xi` are declared as private while any other variables are defaulted to being shared.

1.1.1 Shared access to `Ncon[i]` and `Ncon[j]`

In the parallel region (lines 128 - 149), we encounter a *data race*, particularly in lines 134 and 135, where the shared variable `Ncon` is being updated. The race condition arises because the updates to `Ncon[i]` and `Ncon[j]` depend on their current values, which can be modified concurrently by different threads.

For example, if `thread 0` executes line 135 to update `Ncon[j]`, and at the same time, `thread 1` is executing line 134 with `i = j`, the existing value of `Ncon[i]` (which is `Ncon[j]` in this case) could already have been altered by `thread 0`. This leads to an incorrect increment for `Ncon[i]` in `thread 1`, as the calculation is based on an inconsistent value.

This situation can occur repeatedly across any pair of threads, as multiple threads can simultaneously modify `Ncon[i]` and `Ncon[j]`.

We also encounter a *data race* in lines 144 and 145, where the indices for `Lcon` depend on the values of `Ncon[i]` and `Ncon[j]`. As previously mentioned, `Ncon[i]` and `Ncon[j]` can be incorrectly updated due to the earlier data race. This inconsistency can lead to a subsequent data race in the `Lcon` array, as threads may access and update unintended indices based on these incorrect values. Additionally, we must wrap the if statement on line 138 in the same critical region otherwise it could read an incorrect value of `Ncon[i]` and `Ncon[j]`.

1.1.2 Elimination of the data race

We are instructed with adding additional OpenMP pragmas to eliminate data races. To achieve this, we wrap the lines of code that make references to `Ncon[i]` and `Ncon[j]` with `#pragma omp critical`. This prevents interference between threads that may be attempting to increment and update these values simultaneously.

We now execute the modified code and report the output below.

Threads	Output
32 (default)	Pcon = 0.0004 Av. Num. Clusters. = 840.0000 Av. Largest Cluster = 3949.0000 Time elapsed : 0.0322 seconds
4	Pcon = 0.0004 Av. Num. Clusters. = 825.0000 Av. Largest Cluster = 3985.0000 Time elapsed : 0.0543 seconds

Table 3: Consistent results for parallelised code without data race on two thread settings

We observe no significant change in performance after implementing the modifications. Of course the parallel code runs faster than the unparallel code as shown in Table 1. In terms of reproducibility, the original code experienced data races, which resulted in inconsistent outputs across runs. However, with the new safeguards in place to eliminate these data races, the modified code now produces consistent outputs on each run (the output shown above), confirming its reliability and reproducibility. We created a separate batch script to run our code 50 times and each time produced the consistent output above.

Now, we fix `export OMP_NUM_THREADS=4`.

1.1.3 Baseline Time with Chunk Size $N_{vert}/4$

The baseline time with static scheduling and a chunk size of $N_{vert} / 4$ yields:

Threads	Output
4	Pcon = 0.0004 Av. Num. Clusters. = 825.0000 Av. Largest Cluster = 3985.0000 Time elapsed : 0.0540 seconds

Table 4: Baseline time with static schedule and chunk size $N_{vert} / 4$

This execution time is approximately the same as when we had no specified chunking, at 4 threads. We expect there to be a better schedule for this loop which we try to determine experimentally below.

Threads	Chunk Size	Output
4	Nvert/2	Time elapsed : 0.0721 seconds
4	Nvert/8	Time elapsed : 0.0486 seconds
4	Nvert/16	Time elapsed : 0.0459 seconds
4	Nvert/256	Time elapsed : 0.0435 seconds
4	1 (optimal)	Time elapsed : 0.0431 seconds

Table 5: Different chunks sizes and their corresponding output times averaged over 3 iterations

We observe a pattern where if we take chunk size closer to 1, we get better times. So the optimal chunk size (at 4 threads) is 1.

Next, we change `Np` to 8 and `Ngraphs` to 20. This will take substantially longer to run, hence any tests here on are conducted through the SC RTP job queue for accurate timings.

The time taken for the code to execute, with these parameters on 4 threads, is

Time elapsed : 22.1698 seconds.

1.2 Parallelism over P

To parallelise over the `Np` loop we consider the following pragma:

```
#pragma omp parallel for default(shared) private(ip, igrph, i, j, xi, lclus, nclus,
Ncon, Lcon, Pcon, avlclus, avnclus)
```

Here, we inspect the code to determine our private variables. Firstly, since the pragma is paralling the loop, the variable `ip` is assumed private but we declare it as private anyways. The counter for any other loops must be private so this would include `igrph`, `i` and `j`. We also have summatory variables like `avlclus` and `avnclus` both of which are private and initialised to 0.0 at the start of each iteration. We also initialise `Ncon` and `Lcon` inside of this loop and hence these are private. Since `lclus` and `nclus` are assigned values on each iteration by a subroutine, these should be private to the iteration. `Pcon` is computed exclusively for each iteration so this should be private too as should `xi`. Something like `Ngraphs` is shared since we want to average over the same count of graphs for each iteration. The same applies to `Nvert`, `Maxcon`, `Np` and `Pcon_step`. We are required to time the code on the SC RTP job queue for different threads using `Np = 8` and `Ngraphs = 20`.

Threads	Output
1	Time elapsed : 36.5772 seconds
2	Time elapsed : 22.8610 seconds
3	Time elapsed : 12.5941 seconds
4	Time elapsed : 12.2749 seconds

Table 6: Results of parallelism over P for various thread counts

We conclude that parallelising over the values of P is more efficient than parallelising over the vertices. When using 4 threads, the time taken for vertex-based parallelism was 22.1698 seconds, while parallelism over P reduced the execution time to 12.2749 seconds. The only difference between these two approaches is the loop being parallelised, as all other factors, such as the number of threads (4) and parameters ($N_p = 8$, $N_{graphs} = 20$), were kept constant to ensure a fair comparison.

To make the output ordered, we add a `ordered` clause to our parallel directive over the N_p loop, and then enclose the `printf` statement for the output with a `#pragma omp ordered` construct. This will make the program output ordered as per the serial version. We now repeat our timings like above.

Threads	Output
1	Time elapsed : 37.1415 seconds
2	Time elapsed : 28.9663 seconds
3	Time elapsed : 23.6194 seconds
4	Time elapsed : 19.8996 seconds

Table 7: Results of parallelism over P (ordered outputs) for various thread counts

By introducing the ordering of the output statements, we lose performance as expected and we observe, as the number of threads increases, the differences in timings without ordering increases which is what we expect as more threads means more ordering has to be done between threads thereby reducing performance more.

1.3 Parallelism over graphs

To parallelise over the graphs, we consider the following pragma:

```
pragma omp parallel for default(shared) private(igraph, Ncon, Lcon, lclus, nclus, xi, i, j) reduction(+:avlclus,avnclus)
```

Here, we can have `igraph`, `Ncon`, `Lcon`, `lclus`, `nclus`, `xi`, `i` and `j` are private variables while all others are shared hence we use `default(shared)`. We have two lines that accumulate the averages, specifically `avlclus += (double)lclus;` and `avnclus += (double)nclus;` which can be seen as reduction variables using the addition operation.

We are required to time the code on the SCRTP job queue for different threads using parameters $N_p = 8$ and $N_{graphs} = 20$ for comparability. These are given below:

Threads	Output
1	Time elapsed : 38.5978 seconds
2	Time elapsed : 16.4672 seconds
3	Time elapsed : 11.4185 seconds
4	Time elapsed : 8.3246 seconds

Table 8: Results of parallelism over graphs for various thread counts

This parallelism is more efficient than parallelising over P , we observe that the ratios in speed increase as we increase threads is greater with this method compared to parallelising over P . In fact using 4 threads is more than 2 times faster when parallelising over graphs than over P .

1.3.1 Parallel overheads

We observe that the loop over graphs is a more inner loop than the loop over P which means that the number of forks and joins for the threads is greater when parallelising over graphs than when parallelising over P . This creates more parallel overhead and could decrease performance more.

1.3.2 Distribution of work to threads

We also observe that by parallelising over P , we have more work per thread since this is a more outer loop and contains the averaging over multiple graphs loop. Therefore, in this case the parallelising over P loop could be slower since there is a greater workload per thread.

As mentioned before, by considering the two factors which oppose each other, we conclude that the parallelism over graphs performs better than the parallelism over P , as seen in the data tables provided.

1.4 Nested Parallelism

We now combine parallelism over both P and graphs, while ensuring the correct ordering of output, by implementing nested parallelism. Specifically, we set `Np = 2`, assigning a separate thread to handle each of the two values of P . For each of these threads, we fork into two additional threads to distribute the workload of generating 20 graphs, dividing this task between the two threads for each value of P . This approach effectively balances the computational load across all threads.

We introduce two key lines of code: `omp_set_nested(1)`, which enables nested parallelism, and `omp_set_num_threads(2)`, which sets the number of threads for the outermost loop over P to 2. We apply the previously discussed pragma to the loop over P , and similarly to the loop over graphs. Additionally, we include the `num_threads(2)` clause in both pragmas, ensuring that each level of parallelism forks into 2 threads, resulting in a total of 4 threads. The program outputs the following:

Output
Pcon = 0.0004 Av. Num. Clusters. = 817.6500 Av. Largest Cluster = 3967.4500 Pcon = 0.0008 Av. Num. Clusters. = 101.1500 Av. Largest Cluster = 4895.4500 Time elapsed : 1.3855 seconds

Table 9: Output of code when using nested parallelism

I tried to increase the number of threads on the inner loop to 6 and the resulting time was practically no different to the above suggesting this structure of threading is very efficient. Previously, we computed 8 values of P over 20 graphs in a minimum of 8.3246 seconds when using parallelism over graphs. So if we scale this quantity down by a factor of 4 (since we only have 2 values of P now) then we have approximately 2 seconds expected run time assuming the nested parallelism had the same effect as before, however we achieve a reduced time of 1.3855 seconds suggesting that the nested parallelism is more efficient than having individual parallelisms.